Relatório de Execução

1. Banco de dados utilizado

O ServicoGestao utiliza MySQL como SGBD. O acesso é realizado com TypeORM, configurado em src/infrastructure/database/data-source.ts. As entidades ORM são ClienteOrm, PlanoOrm e AssinaturaOrm.

As principais tabelas do serviço são: clientes, planos e assinaturas. A tabela assinaturas guarda os códigos do cliente e do plano, as datas de fidelidade, a data do último pagamento, o custo final e a descrição da contratação.

O arquivo database.sql acompanha a entrega como alternativa para criação manual da estrutura. Porém, a configuração padrão do projeto utiliza DB_SYNCHRONIZE=true, fazendo com que o TypeORM crie/ajuste as tabelas ao inicializar o serviço.

Obs: O arquivo .env.example deve ser copiado para .env e preenchido com as credenciais do MySQL disponíveis para o ambiente de execução.

2. Tecnologias utilizadas

- Node.js 20+
- npm 10+
- TypeScript 5.8+
- Express 4.21+
- TypeORM 0.3+
- MySQL
- mysql2
- dotenv
- tsx
- Postman
- Arquitetura Limpa e princípios SOLID

3. Passo a passo para executar o sistema

3.1 Pré-requisitos

- Instalar Node.js e npm.
- Disponibilizar um servidor MySQL acessível pela máquina.
- Instalar o Postman para os testes da API.

3.2 Configurar o projeto

```
cd servico-gestao
```

npm install

copy .env.example .env # Windows

ou

cp .env.example .env # Linux/macOS

Depois, edite o arquivo .env com os valores corretos para DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME e, opcionalmente, PORT e DB_SYNCHRONIZE.

Exemplo:

PORT=3000

DB_HOST=localhost

DB_PORT=3306

DB_USER=seu_usuario

DB_PASSWORD=sua_senha

DB_NAME=seu_banco

DB_SYNCHRONIZE=true

3.3 Criar tabelas e dados iniciais

Com o MySQL acessível e o .env configurado, execute o seed: npm run seed

O seed limpa os registros atuais das três tabelas e cria 10 clientes, 5 planos e 5 assinaturas. O script deve ser executado com atenção porque a limpeza dos registros faz parte da rotina de inicialização de dados de teste.

3.4 Iniciar a API

npm run dev

A aplicação fica disponível, por padrão, em: http://localhost:3000

Health check: GET http://localhost:3000/health

Para execução de produção local:

- npm run build
- npm start

3.5 Testar com Postman

Importe o arquivo

amanda_Desenvolvimento_de_Sistemas_backend_Fase-1.postman_collection.json, localizado na

raiz do pacote de entrega. A variável baseUrl da coleção deve apontar para http://localhost:3000.

Método	Endpoint	Objetivo
GET	/gestao/clientes	Listar clientes
GET	/gestao/planos	Listar planos
POST	/gestao/assinaturas	Criar assinatura
PATCH	/gestao/planos/:idPlano	Atualizar custo mensal
GET	/gestao/assinaturas/:tipo	Listar TODOS/ATIVOS/CANCELADOS
GET	/gestao/assinaturascliente/:codcli	Listar por cliente
GET	/gestao/assinaturasplano/:codplano	Listar por plano

3.6 Observações sobre a execução

- A pasta node_modules não faz parte da entrega e deve ser instalada por npm install.
- O arquivo .env deve ser criado localmente a partir do .env.example.
- O seed é destinado a preparar dados para os testes da Fase 1.
- Os endpoints de pagamento e o ServicoPlanosAtivos pertencem à Fase 2 e não são necessários para testar esta entrega.

3.7 Estrutura de arquivos entregue

➔ amanda-desenvol-sistemas-backend-fase-1-entrega/

➔ amanda_Desenvolvimento_de_Sistemas_backend_Fase-1.postman_collection.json
➔ amanda_relatorio.pdf

➔ amanda_relatorioExecucao.pdf

➔ servico-gestao/

➔ .env.example

➔ README.md

➔ database.sql

➔ package.json

➔ tsconfig.json

➔ src/

➔ application/

➔ config/

➔ domain/

➔ infrastructure/
➔ presentation/
➔ seed/

➔ shared/